Arch Linux

Arch Linux is an independently developed, x86-64 general-purpose GNU/Linux distribution that strives to provide the **latest stable versions** of most software by following a rolling release model .

The default installation is a minimal base system , configured by the user to only add what is purposely required.

1 Principles

1.1 Simplicity

Arch Linux defines simplicity as **without unnecessary additions or modifications** . It ships software as released by the original developers— upstream —with minimal distribution-specific downstream changes. Patches not accepted by upstream are avoided, and Arch's downstream patches consist almost entirely of backported bug fixes that are obsoleted by the project's next release.

In a similar fashion, Arch ships the configuration files provided by upstream with changes limited to distribution-specific issues like adjusting the system file paths. It does not add automation features such as enabling a service simply because the package was installed. Packages are only split when compelling advantages exist, such as to save disk space in particularly bad cases of waste.

Arch Linux official packages do not provide system-wide GUI configuration utilities (i.e. there is neither a GUI installation wizard nor a GUI system configuration tool, and Arch as a distribution does not promote GUI tools for system configuration), encouraging users to perform most system configuration from a command-line shell and a text editor .

1.2 Modernity

Arch Linux strives to maintain the latest stable release versions of its software as long as systemic package breakage can be reasonably avoided. It is based on a rolling-release system, which allows a one-time installation with continuous upgrades.

Arch Linux incorporates the latest available kernels as well as features available to GNU/Linux users, including:

- the systemd init system,
- modern file systems ,
- LVM2 ,
- software RAID ,
- udev support,
- initcpio (with mkinitcpio).

Arch Linux does not keep using ancient things if there are modern, future-proof, and better options.

1.3 Pragmatism

Arch is a pragmatic distribution rather than an ideological one— **the principles here are only useful guidelines.** Ultimately, design decisions are made on a case-by-case basis through developer consensus. Evidence-based technical analysis and debate are what matter, not politics or popular opinion.

The large number of packages and build scripts in the various Arch Linux repositories offer free and open source software for those who prefer it, as well as proprietary software packages for those who embrace functionality over ideology.

1.4 User centrality

Whereas many GNU/Linux distributions attempt to be more **user-friendly** , Arch Linux has always been, and shall always remain **user-centric** :

- The distribution is intended to fill the needs of those contributing to it, rather than trying to appeal to as many users as possible.
- It is targeted at the proficient GNU/Linux user, or anyone with a do-it-yourself attitude who is willing to read the documentation, and solve their own problems.

All users are encouraged to participate and contribute to the distribution. Reporting and helping fix bugs is highly valued and patches improving packages or the core projects are very appreciated: Arch's developers are volunteers and active contributors will often find themselves becoming part of that team. *Archers* can freely contribute packages to the Arch User Repository , improve the ArchWiki documentation , provide technical assistance to others or just exchange opinions in the forums , mailing lists , or IRC channels . Arch Linux is the operating system of choice for many people around the globe, and there exist several international communities that offer help and provide documentation in many different languages.

1.5 Versatility

Arch Linux is a general-purpose distribution. Upon installation, only a command-line environment is provided; rather than tearing out unneeded and unwanted packages, the user is offered the ability to build a custom system by choosing among thousands of high-quality packages provided in the official repositories for the x86-64 architecture.

Arch is a rolling-release model backed by pacman , a lightweight, simple and fast package manager that allows for continuously upgrading the entire system with one command. Arch also provides the Arch build system , a ports-like system to make it easy to build and install packages from source, which can also be synchronized with one command. In addition, the *Arch User Repository* contains many thousands of community-contributed PKGBUILD scripts for compiling installable packages from source using the makepkg application. It is also possible for users to build and maintain their own custom repositories with ease.

2 History

The Arch community has grown and matured to become one of the most popular and influential Linux distributions, also testified by the attention and review received over the years.

Apart from select exceptions , Arch developers remain unpaid, part-time volunteers, and there are no prospects for monetizing Arch Linux, so it will remain free in all senses of the

word. Those curious to peruse more detail about Arch's development history can browse the Arch entry in the Internet Archive Wayback Machine and the Arch Linux News Archives .

2.1 The early years

Judd Vinet —a Canadian programmer and occasional guitarist—began developing Arch Linux in early 2001. Its first formal release—Arch Linux 0.1—was on 2002-03-11. Inspired by the elegant simplicity of Slackware , BSD , PLD Linux and CRUX , and yet disappointed with their lack of package management at the time, Vinet built his own distribution on similar principles as those distributions. But, he also wrote a package management program called pacman , to automatically handle package dependency resolution, installation, removal, and upgrades.

2.2 The middle years

The early Arch community grew steadily, as evidenced by this chart of forum posts, users, and bug reports . Moreover, it was from its early days known as an open, friendly, and helpful community .

2.3 Birth of the ArchWiki

On 2005-07-08 the ArchWiki was first set up on the MediaWiki engine.

2.4 The dawning of the age of Aaron Griffin

In late 2007, Judd Vinet retired from active participation as an Arch developer, and smoothly transferred the reins over to American programmer Aaron Griffin , also known as Phrakture.

The Official Arch Linux Logo Contest —see the submissions —took place at the same time.

2.5 Arch Install Scripts

The 2012-07-15 release of the installation image deprecated the menu-driven *Arch Installation Framework* (AIF) in favor of the *Arch Install Scripts* (arch-install-scripts).

2.6 The systemd era

Between 2012 and 2013 the traditional UNIX System V init system was replaced by systemd . [1] [2] [3] [4]

2.7 Drop of i686 support

On 2017-01-25 it was announced that support for the i686 architecture would be phased out due to its decreasing popularity among the developers and the community. By the end of November 2017, all i686 packages were removed from the mirrors.

2.8 Review of Project Leader role and election

At the start of 2020, in a team effort the Arch Linux staff devised a new process for determining future leaders, documented in DeveloperWiki:Project Leader .

As Aaron Griffin had decided to step down from his role, a poll was held to elect a new person to replace him, and on 2020-02-24 its results were published , making the election of Levente Polyak official.

2.9 The GitLab era

In May 2023 , Arch Linux migrated its packaging infrastructure to a self-hosted GitLab instance. Besides internal changes and innovations , this also resulted in splitting the testing repository into core-testing and extra-testing , the staging repository into *core-staging* and *extra-staging* , and finally *community* has been merged into extra .

Several months later, in November 2023 , the old Flyspray bug tracker bugs.archlinux.org was migrated to gitlab.archlinux.org and its collaboration features—issues and merge requests—were opened to the public. For archiving reasons there remains a static copy of the old bug tracker, so that links—for example the randomly picked FS#56716 —are still valid.

Frequently asked questions

1 General

1.1 What is Arch Linux?

See the Arch Linux article.

1.2 Why would I not want to use Arch?

You may **not** want to use Arch, if:

- you do not have the ability/time/desire for a 'do-it-yourself' GNU/Linux distribution.
- you require support for an architecture other than x86_64.
- you take a strong stance on using a distribution which only provides free software as defined by GNU.
- you believe an operating system should configure itself, run out of the box, and include a complete default set of software and desktop environment on the installation media.
- you do not want a rolling release GNU/Linux distribution.
- you are happy with your current OS.

1.3 Why would I want to use Arch?

Because Arch is the best .

1.4 What architectures does Arch support?

Arch only supports the x86_64 (sometimes called amd64) architecture. Support for i686 was dropped in November 2017 [1] .

There are derivative distributions for the i686 architecture [2] and ARM CPUs [3] , each with their own community channels. They are not supported by Arch Linux.

If you wish for Arch to support other architectures, you can help out with existing porting efforts or start your own. See Getting involved#Help porting Arch Linux to other architectures .

1.5 Does Arch follow the Linux Foundation's Filesystem Hierarchy Standard (FHS)?

Arch Linux follows the *file system hierarchy* for operating systems using the systemd service manager. See file-hierarchy(7) for an explanation of each directory along with their designations. In particular, `/bin` , `/sbin` , and `/usr/sbin` are symbolic links to `/usr/bin` , and `/lib` and `/lib64` are symbolic links to `/usr/lib` .

1.6 I am a complete GNU/Linux beginner. Should I use Arch?

If you are a beginner and want to use Arch, you must be willing to invest time into learning a new system, and accept that Arch is designed as a 'do-it-yourself' distribution; it is the user who assembles the system.

Before asking for help, do your own independent research by searching the Web, the forum and the superb documentation provided by the Arch Wiki. *There is a reason these resources were made available to you in the first place.* Many thousands of *volunteered* hours have been spent compiling this excellent information.

See also Arch terminology#RTFM and the Installation guide .

1.7 Is Arch designed to be used as a server? A desktop? A workstation?

Arch is not designed for any particular type of use. Rather, it is designed for a particular type of *user* . Arch targets competent users who enjoy its 'do-it-yourself' nature, and who further exploit it to shape the system to fit their unique needs. Therefore, in the hands of its target user base, Arch can be used for virtually any purpose. Many use Arch on both their desktops and workstations. And of course, archlinux.org, aur.archlinux.org and almost all of Arch's infrastructure runs on Arch.

1.8 I really like Arch, except the development team needs to implement feature X

Get involved , contribute your code/solution to the community. If it is well-regarded by the community and development team, perhaps it will be merged. The Arch community thrives on contribution and sharing of code and tools.

1.9 When will the new release be made available?

Arch Linux releases are simply a live environment for installation or rescue, which include the base meta package and a few other packages . The releases are issued usually in the first half of every month.

1.10 Is Arch Linux a stable distribution? Will I get frequent breakage?

It is *the user* who is ultimately responsible for the stability of their own rolling release system. The user decides when to upgrade, and merges necessary changes when required. If the user reaches out to the community, help is often provided in a timely manner. The difference between Arch and other distributions in this regard is that Arch is truly a 'do-it-yourself' distribution; complaints of breakage are misguided and unproductive, since upstream changes are not the responsibility of Arch devs.

See the System maintenance article for tips on how to make an Arch Linux system as stable as possible.

1.11 Arch needs more press (i.e. advertisement)

Arch gets plenty of press as it is. The goal of Arch Linux is not to be large; rather, organic, sustainable growth occurs naturally amongst the target user base.

1.12 Arch needs more developers

Possibly so. Feel free to volunteer your time! Visit the forums , IRC channels , and mailing lists , and see what needs to be done. See also Getting involved for details.

2 Installation

2.1 Arch needs an installer. Maybe a GUI installer?

A guided installer with a text-based user interface is available. See archinstall for details.

2.2 I installed Arch, and now I am at a shell! What now?

See General recommendations .

2.3 Which desktop environment or window manager should I use?

Since many are available to you, use the one that best fits your needs. Have a look at the Desktop environment and Window manager articles.

2.4 What makes Arch unique amongst other "minimal" distributions?

See Arch compared to other distributions .

3 System maintenance

See also System maintenance .

3.1 Why is my internet so slow compared to other operating systems?

Is your network configured correctly? Have a look at the Network configuration article. For advanced setups, you may also want to look at traffic shaping .

One of the most commonly used kernels, linux , tends to be newer than the kernel of other, more stable, Linux distributions. Because of that, you may rarely experience a kernel regression or driver bug, especially if using Wi-Fi. Do note that the vast majority of those bugs are not Arch Linux-specific as Arch Linux only applies the most basic of patches. This needs to be taken upstream. See #I have found an error with package X. What should I do? .

3.2 Why is Arch using all my RAM?

Essentially, unused RAM is wasted RAM.

Many new users notice how the Linux kernel handles memory differently than they are used to. Since accessing data from RAM is much faster than from a storage drive, the kernel caches recently accessed data in memory. The cached data is only cleared when the system begins to run out of available memory and new data needs to be loaded.

We could distinguish the difference from `free` command:

```
$ free -h
```

```
               total        used        free      shared
buff/cache   available
Mem:            377Gi        40Gi       146Gi
1.1Gi       196Gi       337Gi
Swap:           377Gi       1.1Gi       376Gi
```

It is important to note the difference between "free" and "available" memory. In the above example, a system with 377 GiB of total RAM appears to be using more than half of it, with only 146 GiB as free memory. However, 196 GiB of it is "buff/cache". There is still 337 GiB available for starting new applications, without swapping. See free(1) for details. The result of all this? Performance!

See this wonderful article if your curiosity has been piqued. There is also a website dedicated to clearing this confusion: https://www.linuxatemyram.com/ .

3.3 Where did all my free space go?

The answer to this question depends on your system. There are some fine utilities that may help you find the answer.

3.4 Why am I unable to log in?

Have you mistyped your password or cancelled a sudo command three times in fifteen minutes? If so, you have triggered a prevention mechanism against brute-force attacks: see Security#Lock out user after three failed login attempts for more details.

3.5 Does Arch "phone home"?

In short? No.

In more details:

- Users of NetworkManager should know it does automatic connectivity checks . The default connectivity URL is provided by Arch and committed to not log any access.
- Network Time Protocol clients in their default configuration use a vendor pool of NTP servers provided by the NTP Pool Project (per its rules).
- As explained by the note in pacman/Package signing#Upgrade system regularly , a systemd timer runs once a week to update the new signatures of already trusted keys. There is no logging of any access there either.
- The mirror list updating tool Reflector downloads the mirror list from archlinux.org .

You may want to voluntarily "phone home" by contributing to the pkgstats project that collects anonymous data of package popularity to help Arch developers prioritize their efforts.

4 Package management

See the pacman , pacman/Tips and tricks and Official repositories pages for more answers.

4.1 I have found an error with package X. What should I do?

First, you need to figure out if this error is something the Arch team can fix. Often it is not (e.g. Firefox crashes may be the fault of the Mozilla team); this is called an *upstream error* , see Bug reporting guidelines#Upstream or Arch? . If it is an Arch problem, there is a series of steps you can take:

1. Search the forums for information. See if anyone else has noticed it.
2. Post a bug report with detailed information on the Arch Linux bug tracker in GitLab .
3. If you would like, write a forum post detailing the problem and the fact that you have reported it already. This will help prevent a lot of people from reporting the same error.

4.2 Arch packages need to use a unique naming convention. ".pkg.tar.zst" is too long and/or confusing

This has been discussed on the Arch mailing list. Some proposed a *.pac* file extension, but there is no plan to change the package extension. As Tobias Kieslich, one of the Arch developers, put it, "A package **is** a *[compressed]* tarball! And it can be opened, investigated and manipulated by any tar-capable application. Moreover, the mime-type is automatically detected correctly by most applications."

4.3 Pacman needs a library so other applications can easily access package information

Pacman is a front-end to libalpm(3) —the "Arch Linux Package Management" library— which allows alternative front-ends, like a GUI front-end, to be written.

4.4 Pacman needs feature X!

If you think an idea has merit, you may choose to discuss it on pacman-dev . Also check https://gitlab.archlinux.org/pacman/pacman/-/issues for existing feature requests.

However, the best way to get a feature added to pacman or Arch Linux is to implement it yourself. The patch or code may or may not be officially accepted, but perhaps others will appreciate, test and contribute to your effort.

4.5 I just installed Package X. How do I start it?

If you are using a desktop environment like KDE or GNOME , the program should automatically show up in your menu if it comes with a desktop entry . If you are trying to run the program from a terminal and do not know the binary name, use:

```
$ pacman -Qlq package_name | grep /usr/bin/
```

4.6 Why is there only a single version of each shared library in the official repositories?

Several distributions, such as Debian, have different versions of shared libraries packaged as different packages: `libfoo1` , `libfoo2` , `libfoo3` and so on. In this way it is possible to have applications compiled against different versions of `libfoo` installed on the same system.

In case of a distribution like Arch, only the latest packaged versions are officially supported. By dropping support for outdated software, package maintainers are able to spend more time ensuring that the newest versions work as expected. As soon as a new version of a shared library becomes available from upstream, it is added to the repositories and affected packages are rebuilt to use the new version.

4.7 What if I run a full system upgrade and there will be an update for a shared library, but not for the applications that depend on it?

This scenario should not happen at all. Assuming an application called `foobaz` is in one of the official repositories and builds successfully against a new version of a shared library called `libbaz`, it will be updated along with `libbaz`. If, however, it does not build successfully, `foobaz` package will have a versioned dependency (e.g. *libbaz 1.5*), and will be removed by pacman during `libbaz` upgrade, due to a conflict.

If `foobaz` is a package that you built yourself and installed from AUR, you should rebuild `foobaz` against the new version of `libbaz`. If the build fails, report the bug to the `foobaz` developers.

4.8 Is it possible that there is a major kernel update in the repository, and that some of the driver packages have not been updated?

No, it is not possible. Major kernel updates (e.g. *linux 3.5.0-1* to *linux 3.6.0-1*) are always accompanied by rebuilds of all supported kernel driver packages. On the other hand, if you have an unsupported driver package (e.g. from the AUR) installed on your system, then a kernel update might break things for you if you do not rebuild it for the new kernel. Users are responsible for updating any unsupported driver packages that they have installed.

4.9 What to do before upgrading?

Follow the System maintenance#Upgrading the system section.

4.10 A package update was released, but pacman says the system is up to date

pacman mirrors are not synced immediately. It may take over 24 hours before an update is available to you. The only options are be patient or use another mirror. MirrorStatus can help you identify an up-to-date mirror.

4.11 Upstream project X has released a new version. How long will it take for the Arch package to update to that new version?

Package updates will be released when they are ready. The specific amount of time can be as short as a few hours after upstream releases a minor bugfix update to as long as several weeks after a large package group's major update. The amount of time from an upstream's new version to Arch releasing a new package depends on the specific packages and the availability of the package maintainers. Additionally, some packages spend some time in the testing repository, so this can prolong the time before a package is updated. Package maintainers attempt to work quickly to bring stable updates to the repositories. If you find a package in the official repositories that is out of date, go to that package's page at the package website and flag it.

4.12 If I need an older version of an installed library, can I just symlink to the newer version?

If you are lucky, it might work, for a time. Regardless, it is not a proper solution, because:

- Libraries do not change versions randomly – the API/ABI will have likely changed (possibly with bits removed), and whether those changes affect the usage is just a matter of luck.
- The symlink would be untracked by a package manager. Beginners who immediately try to hack on system library files are in the greatest risk of making an unwanted change that they cannot diagnose/fix, which a package manager helps to guard against.
- A slight alternative of dumping the old library file into the filesystem, untracked, would be forgotten about, and not have potential security bugs noticed/patched.

Instead, e.g. use/write a compat (compatibility) package , which provides the required library version.

5 64-bit

5.1 How do I determine if my processor is x86_64 compatible?

If your processor is x86_64 compatible, you will have the `lm` (long mode) flag in `/proc/cpuinfo` . For example,

```
$ grep -w lm /proc/cpuinfo
```

Under Windows, using the freeware CPU-Z helps determine whether your CPU is 64-bit compatible. CPUs with AMD's instruction set "AMD64" or Intel's solution "EM64T" should be compatible with the x86_64 releases and binary packages.

5.2 Why 64-bit?

It is faster under most circumstances and as an added bonus also inherently more secure due to the nature of Address space layout randomization (ASLR) in combination with Position-independent code (PIC) and the NX Bit which is not available in the stock i686 kernel due to disabled Physical Address Extension (PAE) . If your computer has more than 4 GiB of RAM, only a 64-bit OS will be able to fully utilize it.

Programmers also increasingly tend to care less about 32-bit ("legacy") as "new" x86 CPUs typically support the 64-bit extensions.

There are many more reasons we could list here to tell you to avoid 32-bit, but between the kernel, userspace and individual programs it is simply not viable to list every last thing that 64-bit does much better these days.

Partitioning

From Wikipedia :

Disk partitioning or disk slicing is the creation of one or more regions on secondary storage , so that each region can be managed separately.

An entire disk may be allocated to a single partition, or multiple ones for cases such as dual-booting, maintaining a swap partition, or to logically separate data such as audio and video files. The partitioning scheme is stored in a partition table such as Master Boot Record (MBR) or GUID Partition Table (GPT).

Partition tables are created and modified using one of many partitioning tools . The tools available for Arch Linux are listed in the #Partitioning tools section.

Partitions usually contain a file system directly which is accomplished by creating a file system on (a.k.a. formatting) the partition. Alternatively, partitions can contain LVM , block device encryption or RAID , which ultimately provide device files on which a file system can be placed (or the devices can be stacked further).

Any block device (e.g. disk, partition, LUKS device, LVM logical volume or RAID array) that directly contains a mountable file system is called a volume .

1 Partition table

There are two main types of partition table available. These are described below in the #Master Boot Record (MBR) and #GUID Partition Table (GPT) sections along with a discussion on how to choose between the two. A third, less common alternative is using a partitionless disk, which is also discussed.

Use a partitioning tool to view the partition table of a block device .

> **Tip**
> Run `parted /dev/sdX print` or `fdisk -l /dev/sdX` , where `/dev/sdX` is the block device such as `/dev/sda` for a SATA disk, `/dev/nvme0n1` for a NVMe disk or `/dev/mmcblk0` for an eMMC disk. See Device file#Block device names for more information on block device naming.

1.1 Master Boot Record

The Master Boot Record (MBR) is the first 512 bytes of a storage device. It contains an operating system boot loader and the storage device's partition table. It plays an important role in the boot process under BIOS systems. See Wikipedia:Master boot record#Disk partitioning for the MBR structure.

> **Note**
>
> - The MBR is not located in a partition; it is located at the first sector of the device (physical offset 0), preceding the first partition.

- The boot sector present on a partitionless device or within an individual partition is called a volume boot record (VBR) instead.

1.1.1 Master Boot Record (bootstrap code)

The first 440 bytes of MBR are the **bootstrap code area** . On BIOS systems it usually contains the first stage of the boot loader. The bootstrap code can be backed up, restored from backup or erased using dd .

1.1.2 Master Boot Record (partition table)

In the MBR partition table (also known as DOS or MS-DOS partition table) there are 3 types of partitions:

- Primary
- Extended
  - Logical

Primary partitions can be bootable and are limited to four partitions per disk or RAID volume. If the MBR partition table requires more than four partitions, then one of the primary partitions needs to be replaced by an **extended** partition containing **logical** partitions within it.

Extended partitions can be thought of as containers for logical partitions. A hard disk can contain no more than one extended partition. The extended partition is also counted as a primary partition so if the disk has an extended partition, only three additional primary partitions are possible (i.e. three primary partitions and one extended partition). The number of logical partitions residing in an extended partition is unlimited. A system that dual boots with Windows will require for Windows to reside in a primary partition.

The customary numbering scheme is to create primary partitions *sda1* through *sda3* followed by an extended partition *sda4* . The logical partitions on *sda4* are numbered *sda5* , *sda6* , etc.

Tip
When partitioning an MBR disk consider leaving at least 33 512-byte sectors (16.5 KiB) of free unpartitioned space at the end of the disk in case you ever decide to convert it to GPT . The space will be required for the backup GPT header.

1.2 GUID Partition Table

GUID Partition Table (GPT) is a partitioning scheme that is part of the Unified Extensible Firmware Interface specification; it uses globally unique identifiers (GUIDs), or UUIDs in the Linux world, to define partitions and partition types . It is designed to succeed the Master Boot Record partitioning scheme method.

At the start of a GUID Partition Table disk there is a protective Master Boot Record (PMBR) to protect against GPT-unaware software. This protective MBR just like an ordinary MBR has a bootstrap code area which can be used for BIOS/GPT booting with boot loaders that support it.

1.3 Choosing between GPT and MBR

GUID Partition Table (GPT) is an alternative, contemporary, partitioning style; it is intended to replace the old Master Boot Record (MBR) system. GPT has several advantages over MBR which has quirks dating back to MS-DOS times. With the recent developments to the formatting tools, it is equally easy to get good dependability and performance for GPT or MBR.

> **Note**
> For GRUB or Limine to boot from a GPT-partitioned disk on a BIOS-based system, a BIOS boot partition is required.

Some points to consider when choosing:

- To dual-boot with Windows (both 32-bit and 64-bit) using Legacy BIOS, the MBR scheme is required.
- To dual-boot Windows 64-bit using UEFI mode instead of BIOS, the GPT scheme is required.
- If you are installing on older hardware, especially on old laptops, consider choosing MBR because its BIOS might not support GPT (but see below how to fix it).
- If you are partitioning a disk that is larger than 2 TiB (≈2.2 TB), you need to use GPT.
- It is recommended to always use GPT for UEFI boot, as some UEFI implementations do not support booting to the MBR while in UEFI mode.
- If none of the above apply, choose freely between GPT and MBR. Since GPT is more modern, it is recommended in this case.

Some advantages of GPT over MBR are:

- Provides a unique disk GUID and unique partition GUID (PARTUUID) for each partition – a good filesystem-independent way of referencing partitions and disks. GUIDs are a prerequisite for the Discoverable Partitions Specification that can be utilized in a systemd-enabled initramfs.
- Provides a filesystem-independent partition name (PARTLABEL).
- Arbitrary number of partitions - depends on space allocated for the partition table - No need for extended and logical partitions. By default the GPT table contains space for defining 128 partitions. However if you want to define more partitions, you can allocate more space to the partition table (currently only *gdisk* is known to support this feature).
- Uses 64-bit LBA for storing Sector numbers - maximum addressable disk size is 2 ZiB. MBR is limited to addressing 2 TiB of space per drive. [1]
- Stores a backup header and partition table at the end of the disk that aids in recovery in case the primary ones are damaged.
- CRC32 checksums to detect errors and corruption of the header and partition table.

The section on #Partitioning tools contains a table indicating which tools are available for creating and modifying GPT and MBR tables.

> **Tip**
> It is possible to convert between MBR and GPT. See gdisk#Convert between MBR and GPT .

1.4 Partitionless disk

This article or section needs expansion.

Reason: Explain when one might want to use a partitionless disk (e.g. in VMs) and when not and why. (Discuss in Talk:Partitioning)

Partitionless disk a.k.a. superfloppy refers to a storage device without a partition table, having one file system occupying the whole storage device. The boot sector present on a partitionless device is called a volume boot record (VBR) .

1.4.1 Btrfs partitioning

Btrfs can occupy an entire data storage device and replace the MBR or GPT partitioning schemes. See the Btrfs#Partitionless Btrfs disk instructions for details.

2 Partition scheme

This article or section needs expansion.

Reason: Introduce LVM , mdadm , dm-crypt , etc. They could be placed in a new subsection together with the information about Btrfs subvolumes. (Discuss in Talk:Partitioning)

There are no strict rules for partitioning a hard drive, although one may follow the general guidance given below. A disk partitioning scheme is determined by various issues such as desired flexibility, speed, security, as well as the limitations imposed by available disk space. It is essentially personal preference. If you would like to dual boot Arch Linux and a Windows operating system please see Dual boot with Windows .

> **Note**
>
> - UEFI systems typically need an EFI system partition .
> - BIOS systems that are partitioned with GPT require a BIOS boot partition if GRUB or Limine is used as the boot loader.

> **Tip**
> If using Btrfs , subvolumes can be used to imitate partitions. See the Btrfs#Mounting subvolumes section.

2.1 Single root partition

This scheme is the simplest, most flexible and should be enough for most use cases given the increase in storage size of consumer grade devices. A swap file can be created and easily resized as needed. It usually makes sense to start by considering a single `/` partition and then separate out others based on specific use cases like RAID, encryption, a shared media partition, etc… See #Discrete partitions for a description of some common to uncommon dedicated partitions.

The suggested minimum size is 23–32 GiB for a single root partition. More space may be needed for user files and when using a swap file. A bare minimal installation requires about

2 GiB. As examples, a simple server can fit under 4 GiB while a full KDE Plasma installation uses 10 GiB. Both examples require frequent purges of the package cache .

> **Warning**
> A separate physical (in the main partition table of the disk, not under LVM, software RAID or in a file system subvolume etc.) /boot partition is needed unless your boot loader is capable of accessing the `/boot` directory that resides in `/` . That means the boot loader must have support for everything starting from the block devices, stacked block devices (LVM, RAID, dm-crypt, LUKS, etc.) and ending with the file system on which the kernel(s) and initramfs image(s) reside.

A GPT partition should have the "Linux root (x86-64)" type GUID `4F68BCE3-E8CD-4DB1-96E7-FBCAF984B709` (`8304` type for gdisk). An MBR partition should have the default "Linux" type ID `83` .

2.2 Discrete partitions

Separating out a path as a partition allows for the choice of a different filesystem and mount options. In some cases like a media partition, they can also be shared between operating systems.

Below are some example layouts that can be used when partitioning, and the following subsections detail a few of the directories which can be placed on their own separate partition and then mounted at mount points under `/` . See file-hierarchy(7) for a full description of the contents of these directories.

2.2.1 /

The root directory is the top of the hierarchy, the point where the primary filesystem is mounted and from which all other filesystems stem. All files and directories appear under the root directory `/` , even if they are stored on different physical devices. The contents of the root filesystem must be adequate to boot, restore, recover, and/or repair the system. Therefore, certain directories under `/` are not candidates for separate partitions.

The `/` partition or root partition is necessary and it is the most important. The other partitions can be replaced by it.

> **Warning**
> Directories essential for booting (except for `/boot`) **must** be on the same partition as `/` or mounted in early userspace by the initramfs . These essential directories are: `/etc` and `/usr` [2] .

`/` traditionally contains the `/usr` directory, which can grow significantly depending upon how much software is installed. 15–20 GiB should be sufficient for most users with modern hard disks. If you plan to store a swap file here and do not plan on using a separate /var , you might need a larger partition size (i.e. adding the size of your RAM to be able to hibernate and an additional 8–12 GiB for `/var`).

A GPT partition should have the "Linux root (x86-64)" type GUID `4F68BCE3-E8CD-4DB1-96E7-FBCAF984B709` (`8304` for gdisk). An MBR partition should have the default "Linux" type ID `83` .

2.2.2 /boot

The `/boot` directory contains the vmlinuz and initramfs images as well as the boot loader configuration file and boot loader stages. It also stores data that is used before the kernel begins executing user-space programs. `/boot` is not required for normal system operation, but only during boot and kernel upgrades (when regenerating the initial ramdisk).

Warning

- Your boot loader must be capable of accessing the `/boot` partition. That means the boot loader must have support for everything starting from the block devices, stacked block devices (LVM, RAID, dm-crypt, LUKS, etc.) and ending with the file system on which the kernel(s) and initramfs image(s) reside.
- File systems can get new features not yet supported by boot loaders (e.g. archlinux/packaging/packages/grub#7 , FS#79857 , FS#59047 , FS#58137 , FS#51879 , FS#46856 , FS#38750 , FS#21733 and fscrypt encrypted directories), making them unsuitable for a `/boot` partition unless disabling incompatible features. This can be typically avoided by using FAT32 since it is supported by practically everything and it will not be getting any new features.

See Arch boot process#Boot loader for more information on boot loader requirements and capabilities.

Note
On UEFI systems it is possible to mount the EFI system partition to `/boot` avoiding the need to create another separate partition. See EFI system partition#Mount the partition for more information.

When using an EFI system partition as `/boot` , the requirements are as described in the EFI system partition article—the correct partition type must be set.

In other cases, it is recommended to set the partition type to Extended Boot Loader (XBOOTLDR) Partition which is GPT partition type GUID `BC13C2FF-59E6-4262-A352-B275FD6F7172` (`ea00` type for gdisk , `xbootldr` type for fdisk) or MBR partition type ID `ea` .

In both cases the suggested size for the partition is 1 GiB, which should give enough space to house multiple kernels. If still in doubt, 4 GiB ought to be enough for anybody.

2.2.3 /home

The `/home` directory contains user-specific configuration files, caches, application data and media files.

Separating out `/home` allows `/` to be re-partitioned separately, but note that you can still reinstall Arch with `/home` untouched even if it is not separate—the other top-level directories just need to be removed, and then pacstrap can be run.

You should not share home directories between users on different distributions, because they use incompatible software versions and patches. Instead, consider sharing a media

partition or at least using different home directories on the same `/home` partition. The size of this partition varies.

A GPT partition should have the "Linux home" type GUID `933AC7E1-2EB4-4F13-B844-0E14E2AEF915` (`8302` type for gdisk , `home` type for fdisk). An MBR partition should have the default "Linux" type ID `83` .

2.2.4 Swap

A swap is a file or partition that provides disk space used as virtual memory. Swap files and swap partitions are equally performant, but swap files are much easier to resize as needed. A swap partition can *potentially* be shared between operating systems, but not if hibernation is used.

Since computers have gained memory capacities superior to a gibibit, the previous "twice the amount of physical RAM" rule has become outdated. A sane default size is 4 GiB.

To use hibernation (a.k.a. suspend to disk) it is advised to create the swap partition at the size of RAM. Although the kernel will try to compress the suspend-to-disk image to fit the swap space there is no guarantee it will succeed if the used swap space is significantly smaller than RAM. See Power management/Suspend and hibernate#Hibernation for more information.

A GPT partition should have the "Linux swap" type with GUID `0657FD6D-A4AB-43C4-84E5-0933C84B4F4F` (`8200` type for gdisk , `swap` type for fdisk). An MBR partition should have the "Linux swap" type ID `82` .

2.2.5 /data

One can consider mounting a "data" partition to cover various files to be shared by all users. Using the `/home` partition for this purpose is fine as well. The size of this partition varies.

A GPT partition should have the default "Linux filesystem" type GUID `0FC63DAF-8483-4772-8E79-3D69D8477DE4` . An MBR partition should have the default "Linux" type ID `83` .

2.2.6 /var

The `/var` directory stores variable data such as spool directories and files, administrative and logging data, pacman 's cache, etc. It is used, for example, for caching and logging, and hence frequently read or written. Keeping it in a separate partition avoids running out of disk space due to flunky logs, etc.

It exists to make it possible to mount `/usr` as read-only. Everything that historically went into `/usr` that is written to during system operation (as opposed to installation and software maintenance) must reside under `/var` .

> **Note**
>
> - `/var` contains many small files. The choice of file system type should consider this fact if a separate partition is used.

- Since `/var` is frequently read or written, it is recommended that you consider the location of this partition on a spinning disk .

`/var` will contain, among other data, the pacman cache. Retaining these packages is helpful in case a package upgrade causes instability, requiring a downgrade to an older, archived package. The pacman cache will grow as the system is expanded and updated, but it can be safely cleared if space becomes an issue.

8–12 GiB on a desktop system should be sufficient for `/var` , depending on how much software will be installed. For users of NVIDIA, Wayland and GDM , consider adding to this partition size as to have enough free space to fit your whole video memory .

A GPT partition should have the "Linux variable data" a.k.a. "Linux /var" type GUID `4D21B016-B534-45C2-A9FB-5C16E091FD2D` (`8310` type for gdisk). An MBR partition should have the default "Linux" type ID `83` .

2.3 Example layouts

This article or section needs expansion.

Reason: Improve current examples. (Discuss in Talk:Partitioning#Table draft 2)

The following examples use `/dev/sda` as the example disk with `/dev/sda1` as the first partition. The block device naming scheme will differ if you are partitioning an NVMe disk (e.g. `/dev/nvme0n1` with partitions starting from `/dev/nvme0n1p1`) or an SD card or eMMC disk (e.g. `/dev/mmcblk0` with partitions starting from `/dev/mmcblk0p1`). See Device file#Block device names for more information.

Note

- UEFI booting does not involve any "boot" flag, booting relies solely on the boot entries in NVRAM. Parted and its front-ends use a "boot" flag on GPT to indicate that a partition is an EFI system partition.
- There is no requirement to have all required/wanted partitions on the same disk or to use the same type of partition table for all disks.

2.3.1 UEFI/GPT layout example

Mount point on the installed system	Partition	Partition type GUID	Suggested size
`/boot` or `/efi` [1]	`/dev/sda1`	`C12A7328-F81F-11D2-BA4B-00A0C93EC93B` : EFI system partition	1 GiB
`[SWAP]`	`/dev/sda2`	`0657FD6D-A4AB-43C4-84E5-0933C84B4F4F` : Linux swap	At least 4 GiB or the size of RAM

			to use hibernation
/	/dev/sda3	4F68BCE3-E8CD-4DB1-96E7-FBCAF984B709 : Linux x86-64 root (/)	Remainder of the device. At least 23–32 GiB.

2.3.2 BIOS/MBR layout example

Mount point on the installed system	Partition	Partition type ID	Boot flag	Suggested size
[SWAP]	/dev/sda1	82 : Linux swap	No	At least 4 GiB or the size of RAM to use hibernation
/	/dev/sda2	83 : Linux	Yes	Remainder of the device. At least 23–32 GiB.
N/A	*Unallocated space* [2]	N/A	N/A	At least 16.5 KiB at the end of the disk

2.3.3 BIOS/GPT layout example

Mount point on the installed system	Partition	Partition type GUID	Suggested size
None	/dev/sda1	21686148-6449-6E6F-744E-656564454649: BIOS boot partition [3]	1 MiB
[SWAP]	/dev/sda2	0657FD6D-A4AB-43C4-84E5-0933C84B4F4F : Linux swap	At least 4 GiB or the size of RAM to use hibernation
/	/dev/sda3	4F68BCE3-E8CD-4DB1-96E7-FBCAF984B709 : Linux x86-64 root (/)	Remainder of the device. At least 23–32 GiB.

1. The ESP can be mounted to /efi if the used boot loader is capable of accessing the file system (and everything above it) on which the kernel and initramfs images are

located. See EFI system partition#Typical mount points and the warning in Arch boot process#Boot loader for details.
2. An unpartitioned space of at least 33 512-byte sectors (16.5 KiB) at the end of the disk to allow converting to GPT in the future. The space will be required for the backup GPT header. The recommendation to preserve an unpartitioned space applies to all MBR partitioned disks.
3. A BIOS boot partition is only required when using GRUB or Limine for BIOS booting from a GPT disk, it is not needed when using other boot loaders. The partition has nothing to do with `/boot`, and it must not be formatted with a file system or mounted.

3 Tools

3.1 Partitioning tools

The following programs are used to create and/or manipulate device partition tables and partitions. See the linked articles for the exact commands to be used.

This table will help you to choose utility for your needs:

Name	Package	MBR	GPT	CLI	TUI	Scripting utility
fdisk	util-linux	Yes	Yes	fdisk(8)	cfdisk(8)	sfdisk(8)
GPT fdisk	gptfdisk	No	Yes	gdisk(8)	cgdisk(8)	sgdisk(8)
Parted	parted	Yes	Yes	parted(8)	No	parted(8)

3.1.1 GUI frontends

- **blivet-gui** — Graphical tool for storage configuration. It uses *parted* as backend via python-blivet AUR and libblockdev.

https://github.com/storaged-project/blivet-gui || blivet-gui AUR

- **GNOME Disks** — A GNOME utility for dealing with storage devices. It uses *parted* as backend via udisks2 and libblockdev. Part of gnome.

https://apps.gnome.org/DiskUtility/ || gnome-disk-utility

- **GParted** — Partition editor for graphically managing your disk partitions. It can be used to resize, copy and move partitions without data loss. It uses *parted* as backend.

https://gparted.org/ || gparted

- **KDE Partition Manager** — Utility to help you manage the disks, partitions, and file systems on your computer. It uses *sfdisk* as backend via kpmcore. Part of kde-system.

https://apps.kde.org/partitionmanager/ || partitionmanager

3.2 Backup

- fdisk can create a backup of the partitions table. See fdisk#Backup and restore partition table .
- GPT fdisk can create a binary backup consisting of the protective MBR, the main GPT header, the backup GPT header, and one copy of the partition table. See GPT fdisk#Backup and restore partition table .

3.3 Recovery

- **gpart** — A utility that guesses the contents of a destroyed MBR partition table. Its usage is explained in the gpart(8) man page.

https://github.com/baruch/gpart || gpart

- **GPT fdisk** — A partitioning tool that can restore the primary GPT header (located at the start of the disk) from the secondary GPT header (located at the end of the disk) or vice versa.

https://www.rodsbooks.com/gdisk/ || gptfdisk

- **TestDisk** — A utility that supports recovering lost partitions on both MBR and GPT.

https://www.cgsecurity.org/index.html?testdisk.html || testdisk

4 Partition alignment

The rule of thumb is to align a partition's start and size to mebibytes. See Advanced Format#Partition alignment .

> **Warning**
> Misaligned partitions will prevent being able to use 4096 byte sectors with dm-crypt/LUKS .

5 GPT kernel support

The `CONFIG_EFI_PARTITION` option in the kernel config enables GPT support in the kernel (despite the name, EFI PARTITION which looks close to EFI system partition). This option must be built in the kernel and not compiled as a loadable module. This option is required even if GPT disks are used only for data storage and not for booting. This option is enabled by default in all Arch's officially supported kernels . In case of a custom kernel, enable this option by doing `CONFIG_EFI_PARTITION=y` .

6 Troubleshooting

6.1 Tricking old BIOS into booting from GPT

Some old BIOSes (from before year 2010) attempt to parse the boot sector and refuse to boot it if it does not contain a bootable MBR partition. This is a problem if one wants to use

GPT on this disk, because, from the BIOS viewpoint, it contains only one, non-bootable, MBR partition of type `ee` (i.e., the protective MBR partition). One can mark the protective MBR entry as bootable using `fdisk -t mbr /dev/sda`, and it will work on some BIOSes. However, the UEFI specification prohibits the protective MBR partition entry from being bootable, and UEFI-based boards do care about this, even in the legacy boot mode. So, this matters if one wants to create a GPT-based USB flash drive that is supposed to boot both on modern UEFI-based boards and also on old BIOSes that insist on finding a bootable MBR partition. It is not possible to solve this problem using traditional tools such as fdisk or gdisk, but it is possible to create a fake MBR partition entry suitable for both kinds of BIOSes manually as a sequence of bytes.

The command below will overwrite the second MBR partition slot and add a bootable partition there of type 0 (i.e. unused), covering only the first sector of the device. It will not interfere with the GPT or with the first MBR partition entry which normally contains a protective MBR partition.

```
# printf '\200\0\0\0\0\0\0\0\0\0\0\001\0\0\0' | dd of=/dev/
sda bs=1 seek=462
```

The end result will look like this:

```
# fdisk -t mbr -l /dev/sda
```

```
Disk /dev/sda: 232.9 GiB, 250059350016 bytes, 488397168
sectors
Disk model: ST3250820AS
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: dos
Disk identifier: 0x00000000

Device     Boot Start       End   Sectors   Size Id Type
/dev/sda1           1 488397167 488397167 232.9G ee GPT
/dev/sda2   *       0         0         1   512B  0 Empty

Partition table entries are not in disk order.
```

6.2 Drives are not visible when firmware RAID is enabled

If a SATA or NVMe drive is visible in firmware setup, but not to Linux (e.g. `fdisk -l` does not list it), it is possible that the controller is in firmware RAID mode.

For NVMe, the journal should show something like:

```
kernel: ahci 0000:00:17.0: Found 1 remapped NVMe devices.
kernel: ahci 0000:00:17.0: Switch your BIOS from RAID to AHCI
mode to use them.
```

The solution is to enter firmware setup and disable NVMe RAID mode and change the SATA controller operation mode from *RAID* to *AHCI*. Mind that the setting may have a

different name (e.g. "Intel Rapid Storage Technology", "Intel RST", "Intel VMD controller" or "VMD") and it could also be per-controller or per-port.

> **Warning**
> When dual booting with Windows , preparations need to be made before changing the controller mode. See How to Enable AHCI in Windows 8 and Windows 10 after Installation .

> **Note**
> Some firmware do not have a dedicated NVMe RAID mode setting and, despite the terms not making any sense, use the SATA controller operation mode setting for the NVMe controller too. These firmware simply interpret "SATA operation mode" being set to "AHCI" on NVMe controllers to mean "use native operating mode without firmware RAID". [3] [4] [5]

7 See also

- Wikipedia:Disk partitioning
- Wikipedia:Binary prefix
- Understanding Disk Drive Terminology
- Rod Smith's page on What's a GPT? and Booting OSes from GPT
- Make the most of large drives with GPT and Linux - IBM Developer
- Microsoft's Windows and GPT FAQ
- Partition Alignment (with examples)

Installation guide

This document is a guide for installing Arch Linux using the live system booted from an installation medium made from an official installation image. The installation medium provides accessibility features which are described on the page Install Arch Linux with accessibility options . For alternative means of installation, see Category:Installation process .

Before installing, it would be advised to view the FAQ . For conventions used in this document, see Help:Reading . In particular, code examples may contain placeholders (formatted in `italics`) that must be replaced manually.

This guide is kept concise and you are advised to follow the instructions in the presented order per section. For more detailed instructions, see the respective ArchWiki articles or the various programs' man pages , both linked from this guide. For interactive help, the IRC channel and the forums are also available.

Arch Linux should run on any x86_64 -compatible machine with a minimum of 512 MiB RAM, though more memory is needed to boot the live system for installation. [1] A basic installation should take less than 2 GiB of disk space. As the installation process needs to retrieve packages from a remote repository, this guide assumes a working internet connection is available.

1 Pre-installation

1.1 Acquire an installation image

Visit the Download page and, depending on how you want to boot, acquire the ISO file or a netboot image, and the respective PGP signature.

1.2 Verify signature

It is recommended to verify the image signature before use, especially when downloading from an *HTTP mirror* , where downloads are generally prone to be intercepted to serve malicious images .

Download the *ISO PGP signature* from https://archlinux.org/download/#checksums to the ISO directory and follow the instructions there to verify it.

Alternatively, from an existing Arch Linux installation run:

```
$ pacman-key -v archlinux-version-x86_64.iso.sig
```

Note

- The signature itself could be manipulated if it is downloaded from a mirror site, instead of from archlinux.org . In this case, ensure that the public key, which is

used to decode the signature, is signed by another, trustworthy key. The `gpg` command will output the fingerprint of the public key.
- Another method to verify the authenticity of the signature is to ensure that the public key's fingerprint is identical to the key fingerprint of the Arch Linux developer who signed the ISO-file. See Wikipedia:Public-key cryptography for more information on the public-key process to authenticate keys.

1.3 Prepare an installation medium

The ISO can be supplied to the target machine via a USB flash drive , an optical disc or a network with PXE : follow the appropriate article to prepare yourself an installation medium from the ISO file.

For the netboot image, follow Netboot#Boot from a USB flash drive to prepare a USB flash drive for UEFI booting.

1.4 Boot the live environment

Note
Arch Linux installation images do not support Secure Boot. You will need to disable Secure Boot to boot the installation medium. If desired, Secure Boot can be set up after completing the installation.

1. Point the current boot device to the one which has the Arch Linux installation medium. Typically it is achieved by pressing a key during the POST phase, as indicated on the splash screen. Refer to your motherboard's manual for details.
2. When the installation medium's boot loader menu appears,
   - if you used the ISO, select *Arch Linux install medium* and press `Enter` to enter the installation environment.
   - if you used the Netboot image, choose a geographically close mirror from *Mirror* menu, then select *Boot Arch Linux* and press `Enter` .

   Tip
   - The ISO uses systemd-boot for UEFI and syslinux for BIOS booting. Use respectively `e` or `Tab` to enter the boot parameters . The Netboot image uses iPXE and the boot parameters can be specified in the *Boot options* menu. See /usr/share/doc/mkinitcpio-archiso/README.bootparams for a list.
   - A common example of manually defined boot parameter would be the font size. For better readability on HiDPI screens—when they are not already recognized as such—using `fbcon=font:TER16x32` can help. See HiDPI#Linux console (tty) for a detailed explanation.

3. You will be logged in on the first virtual console as the root user, and presented with a Zsh shell prompt.

To switch to a different console—for example, to view this guide with Lynx alongside the installation—use the `Alt+ arrow` keyboard shortcut . To edit configuration files, mcedit(1) , nano and vim are available. See pkglist.x86_64.txt for a list of the packages included in the installation medium.

1.5 Set the console keyboard layout and font

The default console keymap is US . Available layouts can be listed with:

```
# localectl list-keymaps
```

To set the keyboard layout, pass its name to loadkeys(1) . For example, to set a German keyboard layout:

```
# loadkeys de-latin1
```

Console fonts are located in `/usr/share/kbd/consolefonts/` and can likewise be set with setfont(8) omitting the path and file extension. For example, to use one of the largest fonts suitable for HiDPI screens , run:

```
# setfont ter-132b
```

1.6 Verify the boot mode

To verify the boot mode, check the UEFI bitness :

```
# cat /sys/firmware/efi/fw_platform_size
```

- If the command returns `64` , the system is booted in UEFI mode and has a 64-bit x64 UEFI.
- If the command returns `32` , the system is booted in UEFI mode and has a 32-bit IA32 UEFI. While this is supported, it will limit the boot loader choice to those that support mixed mode booting.
- If it returns `No such file or directory` , the system may be booted in BIOS (or CSM) mode.

If the system did not boot in the mode you desired (UEFI vs BIOS), refer to your motherboard's manual.

1.7 Connect to the internet

To set up a network connection in the live environment, go through the following steps:

1. Ensure your network interface is listed and enabled, for example with ip-link(8) :

   ```
   # ip link
   ```

2. For wireless and WWAN, make sure the card is not blocked with rfkill .
3. Connect to the network:
   - Ethernet—plug in the cable.
   - Wi-Fi—authenticate to the wireless network using iwctl .

    - Mobile broadband modem—connect to the mobile network with the mmcli utility.
4. Configure your network connection:
    - DHCP : dynamic IP address and DNS server assignment (provided by systemd-networkd and systemd-resolved) should work out of the box for Ethernet, WLAN, and WWAN network interfaces.
    - Static IP address: follow Network configuration#Static IP address .
5. The connection may be verified with ping :

```
# ping ping.archlinux.org
```

> **Note**
> In the installation image, systemd-networkd , systemd-resolved , iwd and ModemManager are preconfigured and enabled by default. That will not be the case for the installed system.

1.8 Update the system clock

The live system needs accurate time to prevent package signature verification failures and TLS certificate errors. The systemd-timesyncd service is enabled by default in the live environment and time will be synchronized automatically once a connection to the internet is established.

Use timedatectl(1) to ensure the system clock is synchronized:

```
# timedatectl
```

1.9 Partition the disks

When recognized by the live system, disks are assigned to a block device such as `/dev/sda` , `/dev/nvme0n1` or `/dev/mmcblk0` . To identify these devices, use lsblk or *fdisk* .

```
# fdisk -l
```

Results ending in `rom` , `loop` or `airootfs` may be ignored. `mmcblk*` devices ending in `rpmb` , `boot0` and `boot1` can be ignored.

> **Note**
> If the disk does not show up, make sure the disk controller is not in RAID mode .

> **Tip**
> Check that your NVMe drives and Advanced Format hard disk drives are using the optimal logical sector size before partitioning.

The following partitions are **required** for a chosen device:

- One partition for the root directory `/` .

- For booting in UEFI mode: an EFI system partition .

Use a partitioning tool like fdisk to modify partition tables. For example:

```
# fdisk /dev/the_disk_to_be_partitioned
```

Note

- Take time to plan a long-term partitioning scheme to avoid risky and complicated conversion or re-partitioning procedures in the future.
- If you want to create any stacked block devices for LVM , system encryption or RAID , do it now.
- If the disk from which you want to boot already has an EFI system partition , do not create another one, but use the existing partition instead.
- Swap space can be set on a swap file for file systems supporting it. Alternatively, disk based swap can be avoided entirely by setting up swap on zram after installing the system.

Example layouts

UEFI with GPT

Mount point on the installed system	Partition	Partition type	Suggested size
`/boot` [1]	`/dev/efi_system_partition`	EFI system partition	1 GiB
`[SWAP]`	`/dev/swap_partition`	Linux swap	At least 4 GiB
`/`	`/dev/root_partition`	Linux x86-64 root (/)	Remainder of the device. At least 23–32 GiB.

1. Other mount points , such as `/efi` , are possible, provided that the used boot loader is capable of loading the kernel and initramfs images from the root volume. See the warning in Arch boot process#Boot loader .

BIOS with MBR

Mount point on the installed system	Partition	Partition type	Suggested size
`[SWAP]`	`/dev/swap_partition`	Linux swap	At least 4 GiB
`/`	`/dev/root_partition`	Linux	Remainder of the device. At least 23–32 GiB.

See also Partitioning#Example layouts .

1.10 Format the partitions

Once the partitions have been created, each newly created partition must be formatted with an appropriate file system . See File systems#Create a file system for details.

For example, to create an Ext4 file system on `/dev/ root_partition` , run:

```
# mkfs.ext4 /dev/root_partition
```

If you created a partition for swap , initialize it with mkswap(8) :

```
# mkswap /dev/swap_partition
```

> **Note**
> For stacked block devices replace `/dev/ *_partition` with the appropriate block device path.

If you created an EFI system partition, format it to FAT32 using mkfs.fat(8) .

> **Warning**
> Only format the EFI system partition if you created it during the partitioning step. If there already was an EFI system partition on disk beforehand, reformatting it can destroy the boot loaders of other installed operating systems.

```
# mkfs.fat -F 32 /dev/efi_system_partition
```

1.11 Mount the file systems

Mount the root volume to `/mnt` . For example, if the root volume is `/dev/ root_partition` :

```
# mount /dev/root_partition /mnt
```

Create any remaining mount points under `/mnt` (such as `/mnt/boot` for `/boot`) and mount the volumes in their corresponding hierarchical order.

> **Tip**
> Run mount(8) with the `--mkdir` option to create the specified mount point. Alternatively, create it using mkdir(1) beforehand.

For UEFI systems, mount the EFI system partition. For example:

```
# mount --mkdir /dev/efi_system_partition /mnt/boot
```

If you created a swap volume, enable it with swapon(8) :

```
# swapon /dev/swap_partition
```

genfstab(8) will later detect mounted file systems and swap space.

2 Installation

2.1 Select the mirrors

Packages to be installed must be downloaded from mirror servers , which are defined in `/etc/pacman.d/mirrorlist` . The higher a mirror is placed in the list, the more priority it is given when downloading a package.

On the live system, all HTTPS mirrors are enabled (i.e. uncommented). The topmost worldwide mirror should be fast enough for most people, but you may still want to inspect the file to see if it is satisfactory. If it is not, edit the file accordingly, and move the geographically closest mirrors to the top of the list, although other criteria should be taken into account. Alternatively, you can use reflector to create a mirrorlist file based on various criteria.

This file will later be copied to the new system by *pacstrap* , so it is worth getting right.

2.2 Install essential packages

No configuration (except for `/etc/pacman.d/mirrorlist`) gets carried over from the live environment to the installed system. The only mandatory package to install is base , which **does not** include all tools from the live installation, so installing more packages is frequently necessary.

In particular, review the following software and install everything that you need:

- CPU microcode updates— amd-ucode or intel-ucode —for hardware bug and security fixes,
- userspace utilities for file systems that will be used on the system—for the purposes of e.g. file system creation and fsck ,
- utilities for accessing and managing RAID or LVM if they will be used on the system,
- specific firmware for other devices not included in linux-firmware (e.g. sof-firmware for onboard audio , linux-firmware-marvell for Marvell wireless and any of the multiple firmware packages for Broadcom wireless),
- software necessary for networking (e.g. a network manager or a standalone DHCP client , authentication software for Wi-Fi, ModemManager for mobile broadband connections),
- a console text editor (e.g nano) to allow editing configuration files from the console,
- packages for accessing documentation in man and info pages: man-db , man-pages and texinfo .

For comparison, packages available in the live system can be found in pkglist.x86_64.txt .

To install more packages or package groups, append the names to the pacstrap(8) command below (space separated) or use pacman to install them while chrooted into the new system.

For example, a basic installation with the Linux kernel and firmware for common hardware:

```
# pacstrap -K /mnt base linux linux-firmware
```

Tip

- You can substitute linux with a kernel package of your choice, or you could omit it entirely when installing in a container.
- You could omit the installation of the firmware package when installing in a virtual machine or container.
- This initial package selection in pacstrap only needs to include what is required for the system to boot; all other software can be installed or replaced post-installation.

3 Configure the system

3.1 Fstab

To get needed file systems (like the one used for the boot directory `/boot`) mounted on startup, generate an fstab file with persistent block device naming using genfstab(8). For example, reference file systems by their UUIDs with `-U` :

```
# genfstab -U /mnt >> /mnt/etc/fstab
```

Check the resulting `/mnt/etc/fstab` file, and edit it in case of errors.

3.2 Chroot

To directly interact with the new system's environment, tools, and configurations for the next steps as if you were booted into it, change root into the new system:

```
# arch-chroot /mnt
```

Note
Some systemd tools such as hostnamectl(1), localectl(1) and timedatectl(1) cannot be used inside a chroot(1), as they require an active dbus connection. [2]

3.3 Time

For human convenience (e.g. showing the correct local time or handling Daylight Saving Time), set the time zone:

```
# ln -sf /usr/share/zoneinfo/Area/Location /etc/localtime
```

Run hwclock(8) to generate `/etc/adjtime` :

```
# hwclock --systohc
```

This command assumes the hardware clock is set to UTC . See System time#Time standard for details.

To prevent clock drift and ensure accurate time, set up time synchronization using a Network Time Protocol (NTP) client such as systemd-timesyncd .

3.4 Localization

To use the correct region and language specific formatting (like dates, currency, decimal separators), edit `/etc/locale.gen` and uncomment the UTF-8 locales you will be using. Generate the locales by running:

```
# locale-gen
```

Create the locale.conf(5) file, and set the LANG variable accordingly:

```
/etc/locale.conf
LANG=en_US.UTF-8
```

If you set the console keyboard layout , make the changes persistent in vconsole.conf(5) :

```
/etc/vconsole.conf
KEYMAP=de-latin1
```

3.5 Network configuration

To assign a consistent, identifiable name to your system (particularly useful in a networked environment), create the hostname file:

```
/etc/hostname
yourhostname
```

> **Tip**
> For advice on choosing a hostname, see RFC 1178 . As explained in hostname(7) , it

must contain from 1 to 63 characters, using only lowercase `a` to `z`, `0` to `9`, and `-`, and must not start with `-`.

Complete the network configuration for the newly installed environment. That may include installing suitable network management software, configuring it if necessary and enabling its systemd unit so that it starts at boot.

3.6 Initramfs

Creating a new *initramfs* is usually not required, because mkinitcpio was run on installation of the kernel package with *pacstrap*.

For LVM, system encryption or RAID, modify mkinitcpio.conf(5) and recreate the initramfs image. If you have changed the default console keymap, only recreating the initramfs is required:

```
# mkinitcpio -P
```

3.7 Root password

Set a secure password for the root user to allow performing administrative actions:

```
# passwd
```

3.8 Boot loader

Choose a boot loader applicable to your partitioning scheme and install it. See the explanations and the comparison table in Arch boot process#Boot loader to make your choice, then follow the installation instructions on its dedicated page.

4 Reboot

Exit the chroot environment by typing `exit` or pressing `Ctrl+d`.

Optionally manually unmount all the partitions with `umount -R /mnt`: this allows noticing any "busy" partitions, and finding the cause with fuser(1).

Finally, restart the machine by typing `reboot`: any partitions still mounted will be automatically unmounted by *systemd*. Remember to remove the installation medium and then login into the new system with the root account.

5 Post-installation

See General recommendations for system management directions and post-installation tutorials (like creating unprivileged user accounts, setting up a graphical user interface, sound or a touchpad).

For a list of applications that may be of interest, see List of applications.

General recommendations

This document is an annotated index of popular articles and important information for improving and adding functionalities to the installed Arch system. Readers are assumed to have read and followed the Installation guide to obtain a basic Arch Linux installation. Having read and understood the concepts explained in #System administration and #Package management is *required* for following the other sections of this page and the other articles in the wiki.

1 System administration

This section deals with administrative tasks and system management. See Core utilities and Category:System administration for more.

1.1 Users and groups

A new installation leaves you with only the superuser account, better known as "root". Logging in as root for prolonged periods of time, possibly even exposing it via SSH on a server, is insecure . Instead, you should create and use unprivileged user account(s) for most tasks, only using the root account for system administration. See Users and groups#User management for details.

Users and groups are a mechanism for *access control* ; administrators may fine-tune group membership and ownership to grant or deny users and services access to system resources. Read the Users and groups article for details and potential security risks.

1.2 Security

Read Security for recommendations and best practices on hardening the system.

For a list of applications to allow running commands or starting an interactive shell as another user (e.g. root), see List of applications/Security#Privilege elevation . For graphical equivalents (and how to avoid them), see Privilege elevation for graphical applications .

> **Tip**
> More fine-grained privilege elevation is provided by polkit , which requires additional setup for graphical environments.

1.3 Service management

Arch Linux uses systemd as the init process, which is a system and service manager for Linux. For maintaining your Arch Linux installation, it is a good idea to learn the basics about it.

Interaction with *systemd* is done through the *systemctl* command. See systemd#Basic systemctl usage for more information.

A logging system is also provided, with the command *journalctl* . See journal for more information.

1.4 System maintenance

Arch is a rolling release system and has rapid package turnover, so users have to take some time to do system maintenance .

2 Package management

This section contains helpful information related to package management. See FAQ#Package management and Category:Package management for more.

> **Note**
> It is imperative to keep up to date with changes in Arch Linux that require manual intervention **before** upgrading your system. Subscribe to the arch-announce mailing list or the recent news RSS feed . Alternatively, check the front page Arch news every time before you update.

2.1 pacman

pacman is the Arch Linux *pac* kage *man* ager: it is highly encouraged to become familiar with it before reading any other articles.

For long term handling of cached packages, see pacman#Cleaning the package cache .

See pacman/Tips and tricks for suggestions on how to improve your interaction with *pacman* and package management in general.

2.2 Repositories

See the Official repositories article for details about the purpose of each officially maintained repository.

If you plan on using 32-bit applications, you will want to enable the multilib repository.

The Unofficial user repositories article lists several other unsupported repositories.

You may consider installing the pkgstats service.

2.3 Mirrors

Visit the Mirrors article for steps on taking full advantage of using the fastest and most up to date mirrors of the official repositories. As explained in the article, a particularly good advice is to routinely check the Mirror Status page for a list of mirrors that have been recently synced. This can be automated with Reflector .

2.4 Arch Build System

Ports is a system initially used by BSD distributions consisting of build scripts that reside in a directory tree on the local system. Simply put, each port contains a script within a directory intuitively named after the installable third-party application.

The Arch build system offers the same functionality by providing build scripts called PKGBUILDs , which are populated with information for a given piece of software: integrity hashes, project URL, version, license and build instructions. These PKGBUILDs are

parsed by makepkg , the actual program that generates packages that are cleanly manageable by *pacman* .

Every package in the repositories along with those present in the AUR are subject to recompilation with *makepkg* .

2.5 Arch User Repository

While the Arch Build System allows the ability of building software available in the official repositories, the Arch User Repository (AUR) is the equivalent for user submitted packages. It is an unsupported repository of build scripts accessible through the web interface or through the Aurweb RPC interface .

3 Booting

This section contains information pertaining to the boot process. An overview of the Arch boot process can be found at Arch boot process . See Category:Boot process for more.

3.1 Hardware auto-recognition

Hardware should be auto-detected by udev during the boot process by default. A potential improvement in boot time can be achieved by disabling module auto-loading and specifying required modules manually, as described in Kernel modules . Additionally, Xorg should be able to auto-detect required drivers using `udev` , but users have the option to configure the X server manually too.

3.2 Microcode

Processors may have faulty behaviour , which the kernel can correct by updating the *microcode* on startup. See Microcode for details.

3.3 Retaining boot messages

Once the login prompt appears, the messages from boot are cleared, leaving users unable to gather feedback from them. Disable clearing of boot messages to overcome this limitation.

3.4 Num Lock activation

Num Lock is a toggle key found in most keyboards. For activating Num Lock's number key-assignment during startup, see Activating numlock on bootup .

4 Graphical user interface

This section provides orientation for users wishing to run graphical applications on their system. See Category:Graphical user interfaces for additional resources.

4.1 Display server

This article or section is a candidate for merging with List of applications/Other .

Notes:

- An overview of software does not belong in a "recommendations" article. Make a new section before List_of_applications/Other#Display_managers .
- XLibre quality is questionable—see e.g. https://gitlab.freedesktop.org/xorg/xserver/-/work_items/1760#note_2631460 and https://gitlab.freedesktop.org/xorg/xserver/-/work_items/1797#note_2803526 .

(Discuss in Talk:General recommendations)

Xorg is the public, open-source implementation of the X Window System (commonly X11, or X). It is used for running applications with graphical user interfaces (GUIs).

XLibre is a fork of Xorg that is tear free by default, supports client isolation and fixes several upstream bugs. See the pinned comment of xlibre-xserver [AUR] for details.

Wayland is a newer, alternative display server protocol with several compositors to choose from. Its advantages over Xorg are enhanced security features, more efficient handling of modern graphics tasks and active development while retaining compatibility through Xwayland .

4.2 Display drivers

The default *modesetting* display driver will work with most video cards, but performance may be improved and additional features harnessed by installing the appropriate driver for AMD , Intel or NVIDIA products.

4.3 Desktop environments

Although the display server provides the basic framework for building a graphical environment, additional components may be considered necessary for a complete user experience. Desktop environments such as KDE , GNOME , COSMIC , Xfce , Cinnamon , LXDE , bundle together a wide range of well-integrated applications, such as a window manager or compositor, panel/taskbar, file manager, terminal emulator, text editor, icons, and other utilities. Users with less experience may wish to install a desktop environment for a more familiar environment. See Category:Desktop environments for additional resources.

4.4 Window managers or compositors

A full-fledged desktop environment provides a complete and consistent graphical user interface, but tends to consume a good amount of system resources. Users seeking to maximize performance or otherwise simplify their environment may opt to install a window manager or compositor alone and hand-pick desired extras. Using Xorg , most desktop environments allow use of an alternative window manager as well. Dynamic , stacking , and tiling window managers differ in their handling of window placement.

4.5 Display manager

Most desktop environments include a display manager for automatically starting the graphical environment and managing user logins. Users without a desktop environment can install one separately. Alternatively you may start X at login as a simple alternative to a display manager.

4.6 User directories

Well-known user directories like Downloads or Music are created by the `xdg-user-dirs-update.service` user service, that is provided by xdg-user-dirs and enabled by default upon install. If your desktop environment or window manager does not pull in the package, you can install it and run `xdg-user-dirs-update` manually as per XDG user directories#Creating default directories .

5 Power management

This section may be of use to laptop owners or users otherwise seeking power management controls. See Category:Power management for more.

See Power management for more general overview.

5.1 ACPI events

Users can configure how the system reacts to ACPI events such as pressing the power button or closing a laptop's lid. For the recommended method using systemd , see Power management#ACPI events . For the old method, see acpid .

5.2 CPU frequency scaling

Modern processors can decrease their frequency and voltage to reduce heat and power consumption. Less heat leads to more quiet system and prolongs the life of hardware. See CPU frequency scaling for details.

5.3 Laptops

For articles related to portable computing along with model-specific installation guides, please see Category:Laptops . For a general overview of laptop-related articles and recommendations, see Laptop .

5.4 Suspend and hibernate

See the main article: Power management/Suspend and hibernate .

6 Multimedia

Category:Multimedia includes additional resources.

6.1 Sound system

ALSA is a kernel sound system that should work out the box (it just needs to be unmuted). Sound servers such as PipeWire and PulseAudio can offer additional features and support more complex audio configuration.

See Professional audio for advanced audio requirements.

7 Networking

This section is confined to small networking procedures. See Network configuration for a full configuration guide and Category:Networking for related articles.

7.1 DNS security

For better security while browsing the web, paying online, connecting to SSH services and similar tasks consider using DNSSEC -enabled DNS resolver that can validate signed DNS records, and an encrypted protocol such as DNS over TLS , DNS over HTTPS or DNSCrypt . See Domain name resolution for details.

7.2 Setting up a firewall

A firewall can provide an extra layer of protection on top of the Linux networking stack. While the stock Arch kernel is capable of using Netfilter 's iptables and nftables , neither are enabled by default. It is highly recommended to set up some form of firewall. See Category:Firewalls for available guides.

7.3 Network shares

To share files among the machines in a network, follow the NFS or the SSHFS article.

Use Samba to join a Windows network. To configure the machine to use Active Directory for authentication, read Active Directory integration .

See also Category:Network sharing .

8 Input devices

This section contains popular input device configuration tips. See Category:Input devices for more.

8.1 Keyboard layouts

Non-English or otherwise non-standard keyboards may not function as expected by default. The necessary steps to configure the keymap are different for virtual console and Xorg , they are described in Keyboard configuration in console and Keyboard configuration in Xorg respectively.

8.2 Mouse buttons

Owners of advanced or unusual mice may find that not all mouse buttons are recognized by default, or may wish to assign different actions for extra buttons. Instructions can be found in Mouse buttons .

8.3 Laptop touchpads

Many laptops use Synaptics or ALPS "touchpad" pointing devices. For these, and several other touchpad models, you can use either the Synaptics input driver or libinput; see Touchpad Synaptics and libinput for installation and configuration details.

8.4 TrackPoints

See the TrackPoint article to configure your TrackPoint device.

9 Optimization

This section aims to summarize tweaks, tools and available options useful to improve system and application performance.

9.1 Benchmarking

Benchmarking is the act of measuring performance and comparing the results to another system's results or a widely accepted standard through a unified procedure.

9.2 Improving performance

The Improving performance article gathers information and is a basic rundown about gaining performance in Arch Linux.

9.3 Solid state drives

The Solid state drive article covers many aspects of solid state drives, including configuring them to maximize their lifetimes, e.g. with TRIM .

10 System services

This section relates to daemons .

10.1 File index and search

Most distributions have a *locate* command available to be able to quickly search files. Arch Linux provides several alternatives, see locate for details.

Desktop search engines provide a similar service, while better integrated into desktop environments .

10.2 Local mail delivery

A default setup does not provide a way to synchronize mail. A list of mail delivery agents is available in the Mail server article.

10.3 Printing

CUPS is a standards-based, open source printing system developed by OpenPrinting for Linux. See Category:Printers for printer-specific articles.

11 Appearance

This section contains frequently-sought "eye candy" tweaks for an aesthetically pleasing Arch experience. See Category:Eye candy for more.

11.1 Fonts

You may wish to install a set of TrueType fonts, as only unscalable bitmap fonts are included in a basic Arch system. There are several general-purpose font families providing large Unicode coverage and even metric compatibility with fonts from other operating systems.

A plethora of information on the subject can be found in the Fonts and Font configuration articles.

If spending a significant amount of time working from the virtual console (i.e. outside an X server), users may wish to change the console font to improve readability; see Linux console#Fonts .

11.2 GTK and Qt themes

A big part of the applications with a graphical interface for Linux systems are based on the GTK or the Qt toolkits. See those articles and Uniform look for Qt and GTK applications for ideas to improve the appearance of your installed programs and adapt it to your liking.

12 Console improvements

This section applies to small modifications that improve console programs' practicality. See Category:Command-line shells for more.

12.1 Tab-completion enhancements

It is recommended to properly set up extended tab completion right away, as instructed in the article of your chosen shell .

12.2 Aliases

Aliasing a command, or a group thereof, is a way of saving time when using the console. This is especially helpful for repetitive tasks that do not need significant alteration to their parameters between executions. Common time-saving aliases can be found in Bash#Aliases , which are easily portable to zsh as well.

12.3 Alternative shells

Bash is the shell installed by default in an Arch system. The live installation media, however, uses zsh with the grml-zsh-config addon package. See Command-line shell#List of shells for more alternatives.

12.4 Bash additions

A list of miscellaneous Bash settings, history search and Readline macros is available in Bash#Tips and tricks .

12.5 Colored output

This section is covered in Color output in console .

12.6 Compressed files

Compressed files, or archives, are frequently encountered on a GNU/Linux system. Tar is one of the most commonly used archiving tools, and users should be familiar with its syntax (Arch Linux packages, for example, are simply zstd compressed tarballs). See Archiving and compression .

12.7 Console prompt

The console prompt (`PS1`) can be customized to a great extent. See Bash/Prompt customization or Zsh#Prompts if using Bash or Zsh, respectively.

12.8 Emacs shell

Emacs is known for featuring options beyond the duties of regular text editing, one of these being a full shell replacement. Consult Emacs#Colored output issues for a fix regarding garbled characters that may result from enabling colored output.

12.9 Mouse support

Using a mouse with the console for copy-paste operations can be preferred over GNU Screen 's traditional copy mode. Refer to General purpose mouse for comprehensive directions. Note that you can already do this in terminal emulators with the clipboard .

12.10 Session management

Using terminal multiplexers like tmux or GNU Screen , programs may be run under sessions composed of tabs and panes that can be detached at will, so when the user either kills the terminal emulator, terminates X , or logs off, the programs associated with the session will continue to run in the background as long as the terminal multiplexer server is active. Interacting with the programs requires reattaching to the session.